

INSIDE THE APK HACKING ANDROID FOR CUES AND EXPLOITS



BY
JACK SESSIONS

ROOT://_WHO I AM

- Heckling allowed :)
- Mobile Security & Counterintel Researcher (and professional memelord)
- Bartender IRL, Hacker in VR, and working in AR/AXI
- CyberSec YouTuber @JackSessions 57 subs
- CyberSec Rapper (retired... for now)
- Pre Seed Money Comp Sci Student
- Wanted for Treason... (Kim did not like my rap)
- 2x CVE-2025-50861, CVE-2025-50862

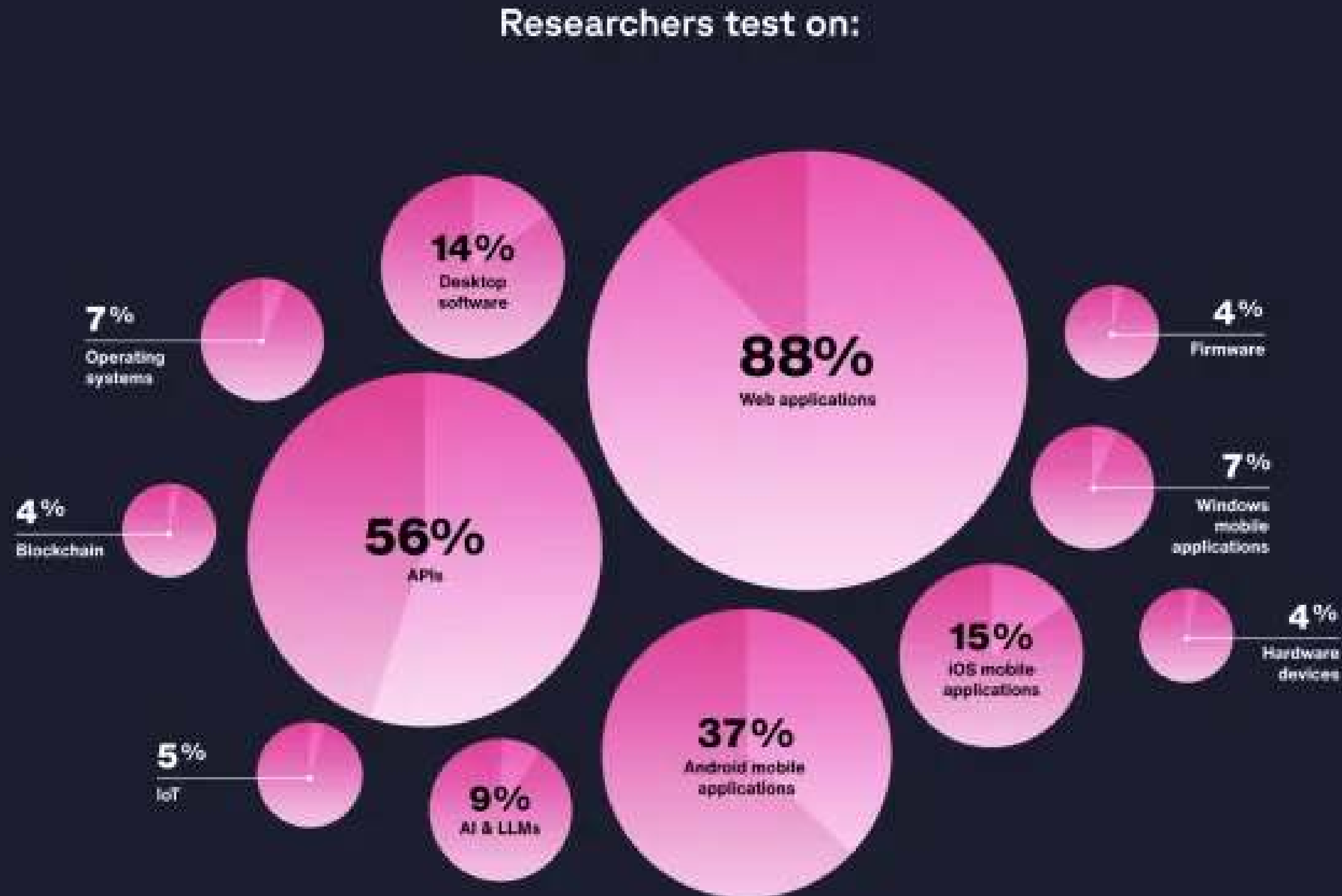


nobody:
hackers on shutterstock:



WHAT YOU'LL LEARN

How And
Why fr
Hidden
Real w
Why And



WHY ANDROID STILL BLEEDS

“Open” doesn’t mean “secure”

Fragmentation = delayed patches

Vendor skins weaken core defenses

Rooting culture → deeper exposure

ADF, FRP, Magisk... attackers love it all



THREAT MODEL

Adversaries

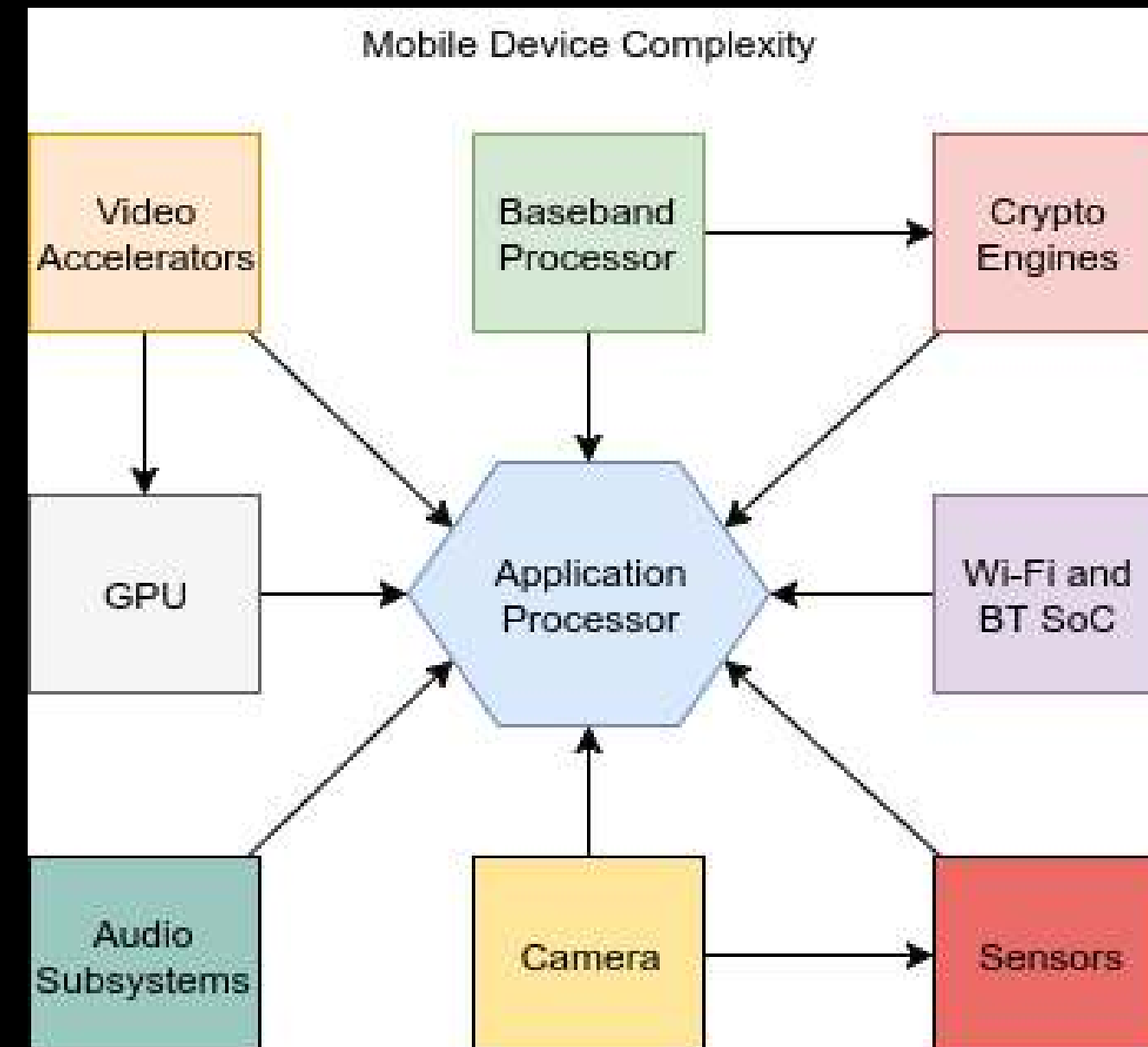
Motivation

Assets at risk

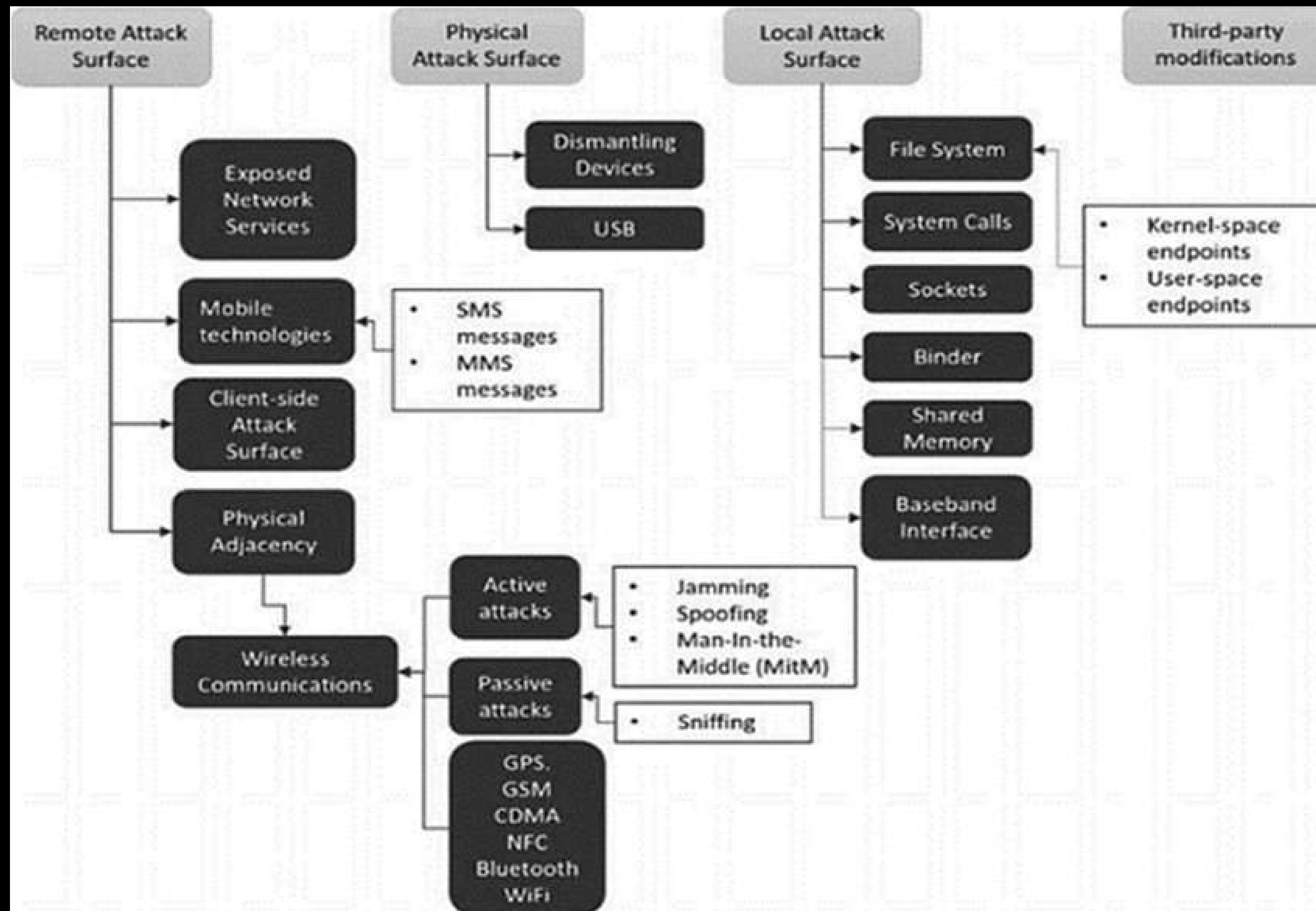
Entry vectors

Weaknesses exploited

Mitigations



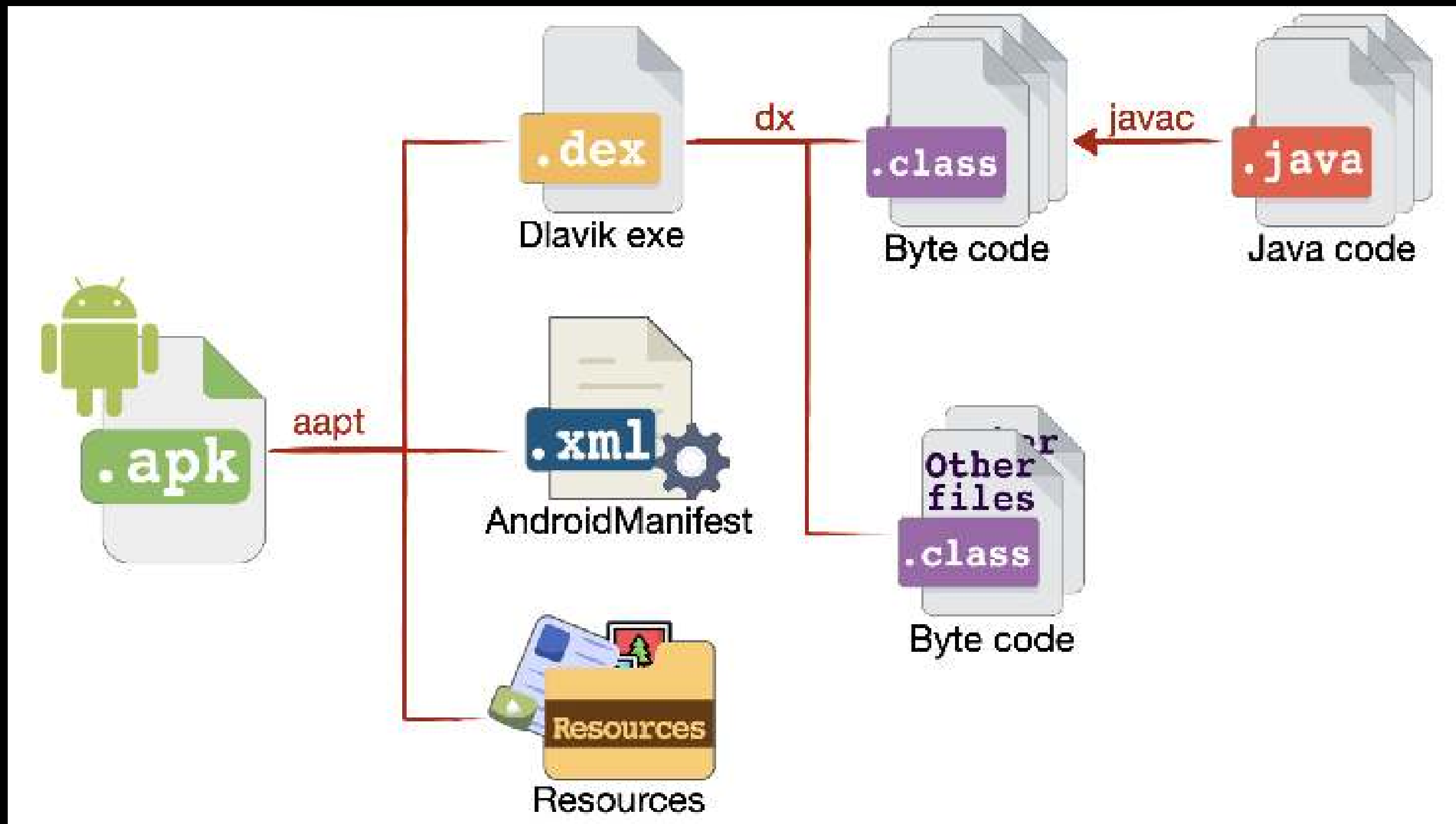
THE APK AS AN ATTACK SURFACE



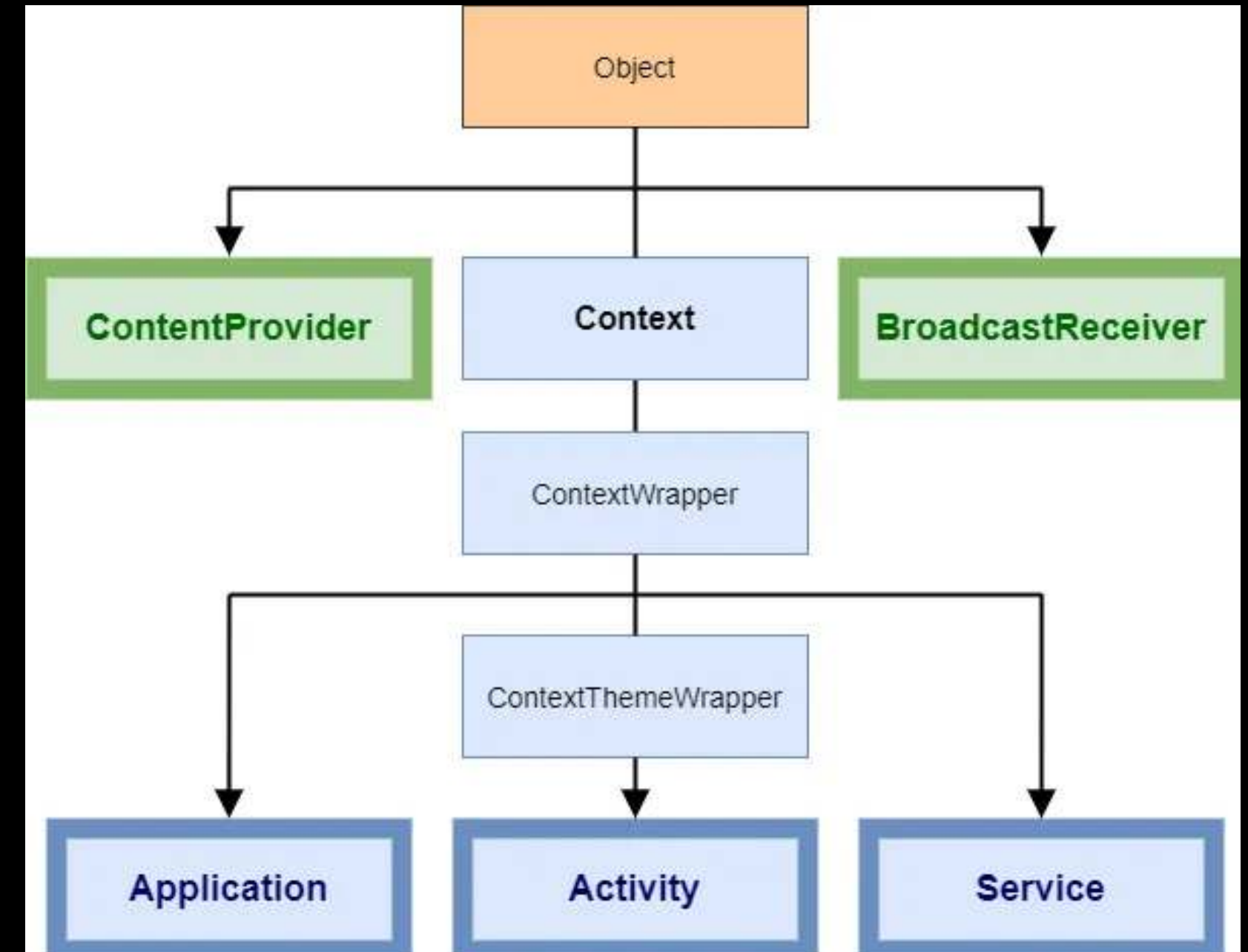
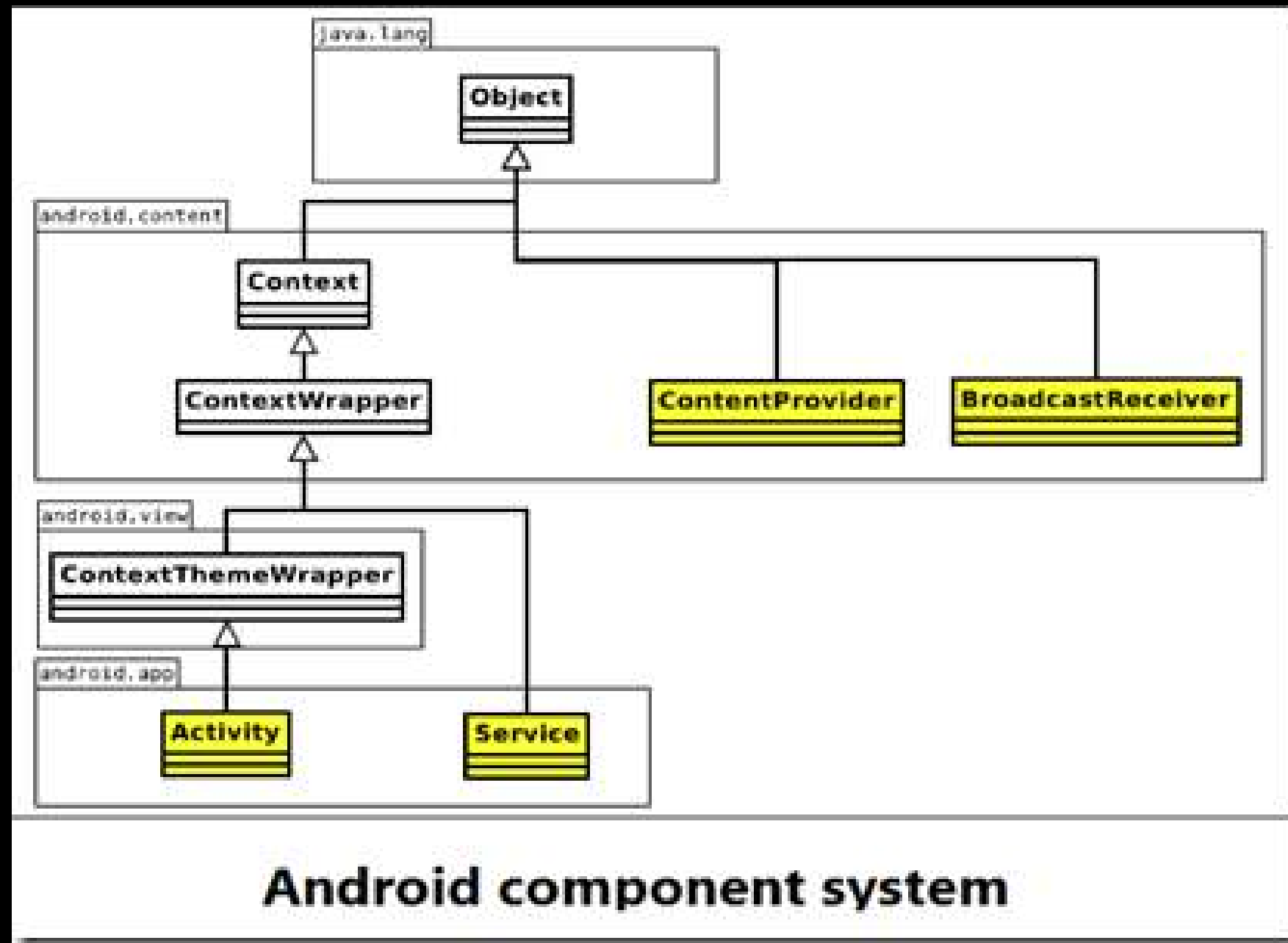
SECTION 2

STATIC ANALYSIS

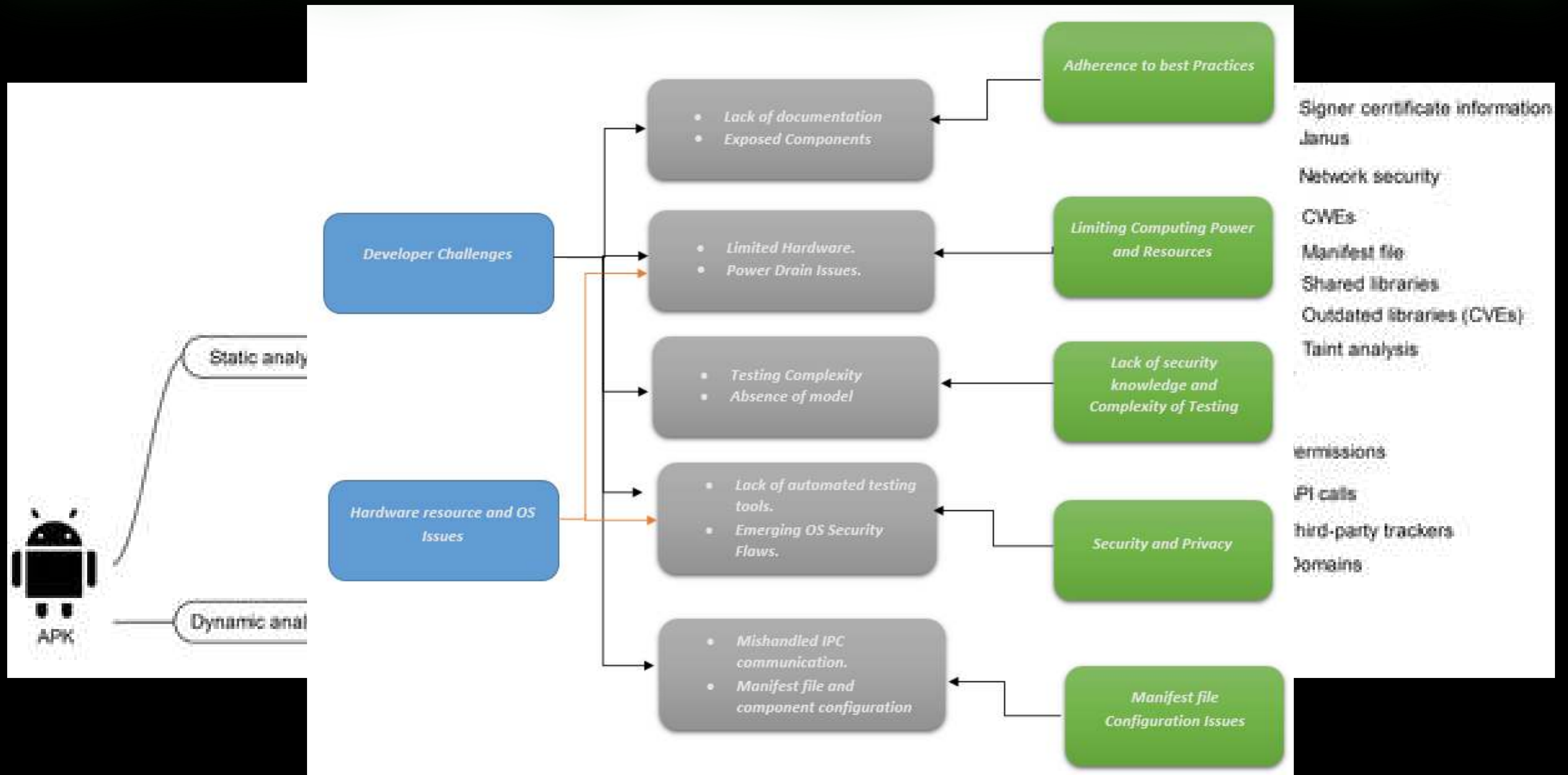
APK ANATOMY



EXPORTED COMPONENTS 101



COMMON MISTAKES IN MANIFESTS



TOOLCHAIN OVERVIEW

Decompile → Inspect → Inject → Repack → Deploy

**Every attacker's first step is understanding the code
and every defender's job is making that painful.**

DEMO #1

EXPOSED SERVICES & BUSINESS

DEMO TAKEAWAY



TRIAGE MATRIX

privilege level × data exposure

Vectors: component type (Service/Receiver/Provider/Activity)

Indicators: exported = true, missing permission, implicit intent

Tools: MobSF static scan + Frida dynamic hooks

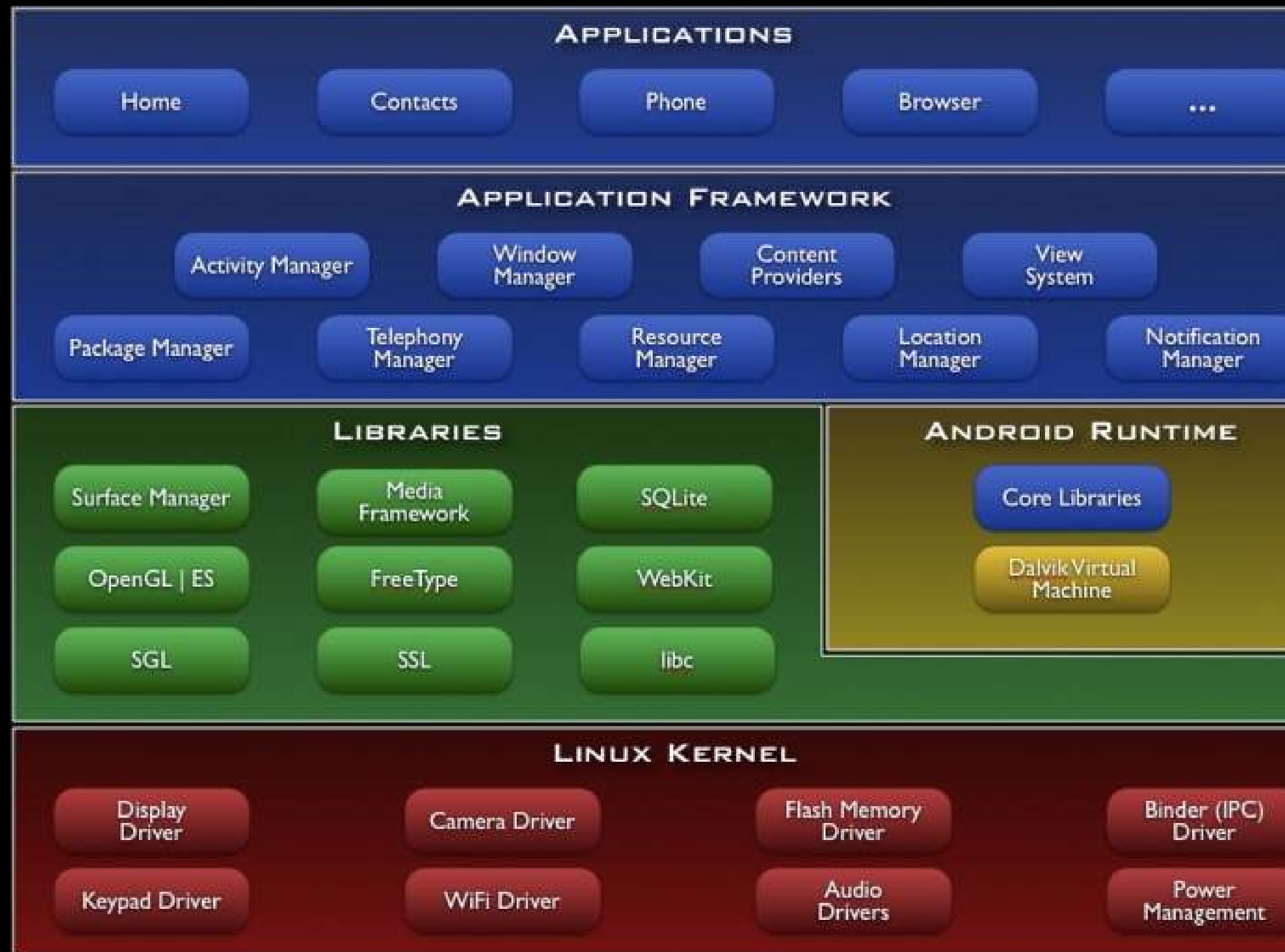
Fix priority: user facing > background > debug

TL;DR: treat exported components like open ports AND scan them first.

SECTION 3

NATIVE LIBRARY & JNI LAYER

ANDROID NATIVE SURFACE



WHY THIS MATTERS

JNI connects Java ↔ Native (C/C++) — boundary = risk

Memory corruption lives below Android permissions

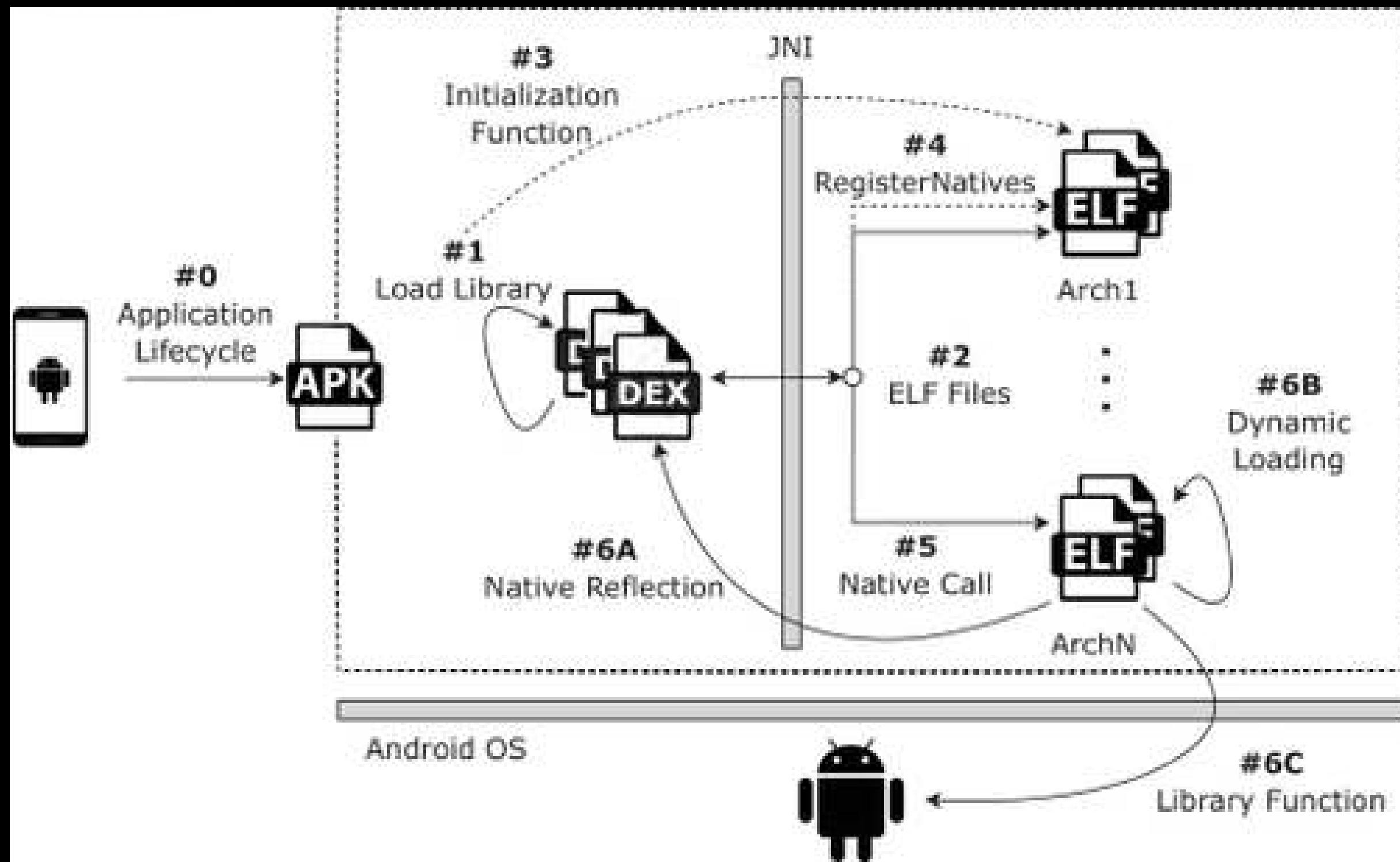
Attackers love libnative.so, libstagefright.so, etc.

Exploits chain from application level → native → kernel

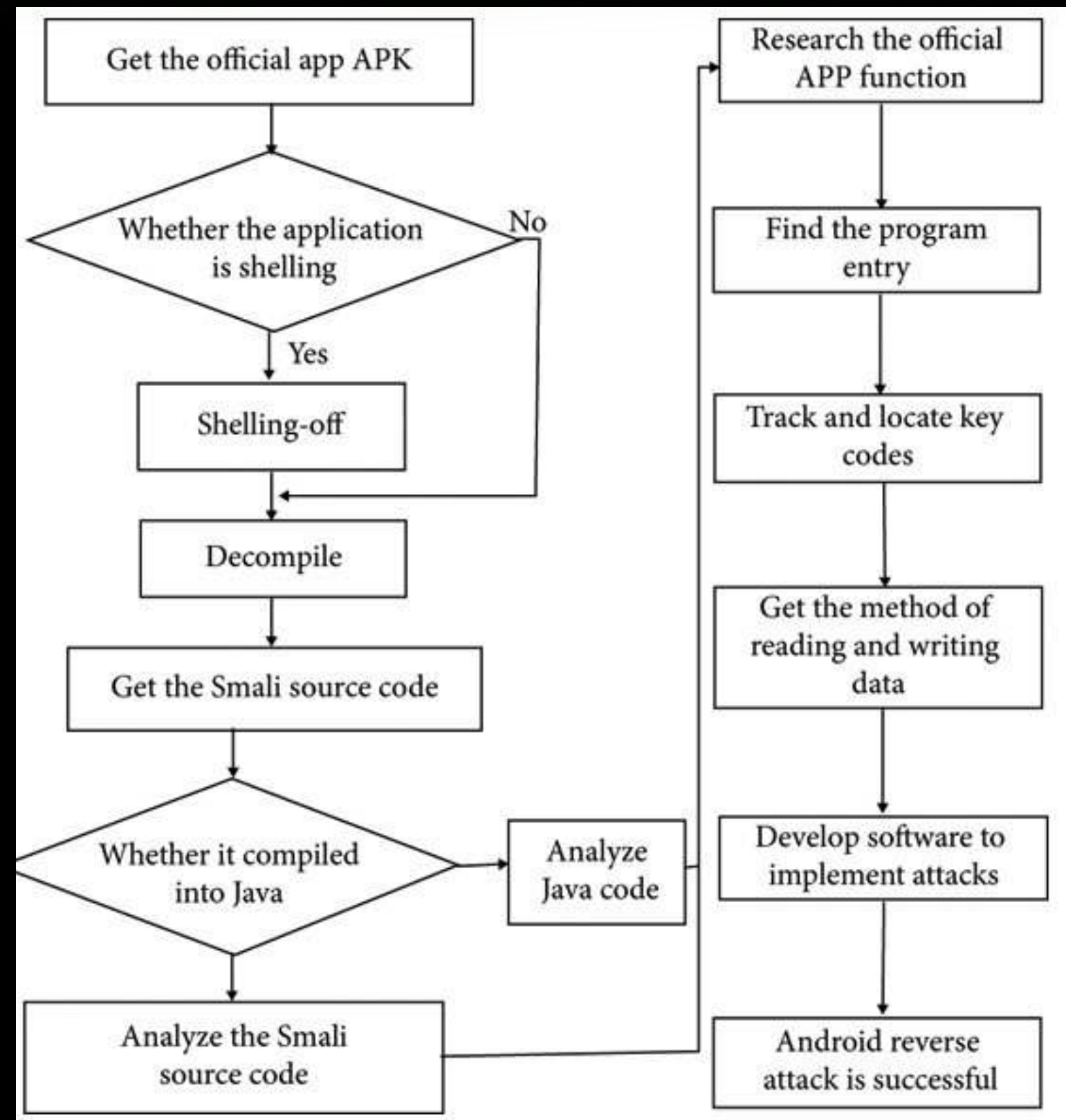
Logs + crash traces expose vulnerable native calls

Understanding JNI flow = finding real zero-days

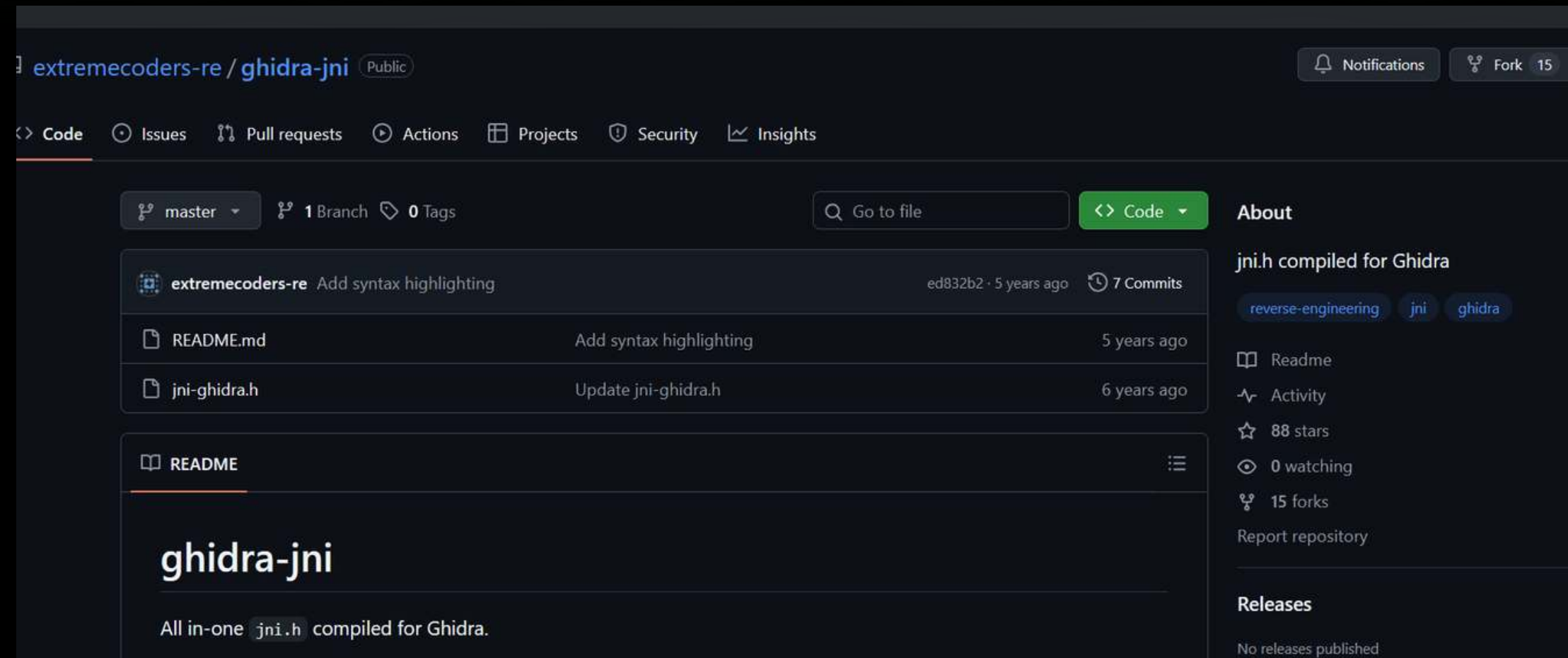
COMMON BUG ISSUES



REVERSE ENGINEERING JNI FUNCTIONS



REVERSE ENGINEERING JNI FUNCTIONS



CRASH TRIAGE 101

Logcat

☐ GIONEE M5 Mini Android 6.0, A ☐ com.example.finapp (715) [DEAD] ☐ Error ☐ Q- ☑ Regex ☐ Show only selected application

logcat

```
06-09 19:38:28.741 715-715/com.example.finapp E/AndroidRuntime: FATAL EXCEPTION: main
Process: com.example.finapp, PID: 715
java.lang.SecurityException: Permission Denial: starting Intent { act=android.intent.action.CALL dat=tel:xxx cmp=com.android.server.telecom/.components/.UserCallActivity } from ProcessRecord{a0b4cce 715:com.example.finapp/u0a327} (pid=715, uid=10327) requires android.permission.CALL_PHONE
at android.os.Parcel.readException(Parcel.java:1599)
at android.os.Parcel.readException(Parcel.java:1552)
at android.app.ActivityManagerProxy.startActivity(ActivityManagerNative.java:2787)
at android.app.Instrumentation.execStartActivity(Instrumentation.java:1518)
at android.app.Activity.startActivityForResult(Activity.java:3990)
at androidx.fragment.app.FragmentActivity.startActivityForResult(FragmentActivity.java:767)
at android.app.Activity.startActivityForResult(Activity.java:3931)
at androidx.fragment.app.FragmentActivity.startActivityForResult(FragmentActivity.java:754)
at android.app.Activity.startActivity(Activity.java:4281)
at android.app.Activity.startActivity(Activity.java:4249)
at com.example.finapp.MainActivity.makeCall(MainActivity.java:70)
at com.example.finapp.MainActivity.access$000(MainActivity.java:37)
at com.example.finapp.MainActivity$1.onClick(MainActivity.java:57)
at android.view.View.performClick(View.java:5272)
at android.view.View$PerformClick.run(View.java:21563)
```

Logcat Profiler Terminal Build Run TODO Event Log

Sign out

[Tracks] - Ping tracking service
Oct 20, 2021 at 12:43:07 PM EDT

DEMO #2

APK → CVE WORKFLOW

SECTION 4

EXPLOIT TRIAGE & CVE PROCESS

CVE WORKFLOW OVERVIEW



STORY TIME!

From   donut.script@proton.me
To JackSessions@protonmail.com

☆ Sep 2

Hi Jack,

Thank you for your response.

In this case, the vulnerable application is an internal tool so it is not publicly distributed, which I think doesn't qualify it for a CVE or that MITRE could handle disclosure to Lotus.

Public disclosure isn't possible either, the severity is very high.

If you are in direct contact with them, please let them know I can access any employee account.

I appreciate that you cc'ed the Lotus team. I'll wait to hear back from them, and in the meantime I will reach out to their legal team.

Best regards,

François

YOUR TRIAGE LADDER

Start with what crashes AND not what looks cool!

Static → Dynamic → Runtime → Context

Rank impact, not noise

Focus on reproducibility, not speculation

Every bug teaches you something

MITIGATIONS FOR DEFENDERS

Enforce least privilege across apps & services

Monitor for abnormal process behavior

Secure exported components & IPC surfaces

Validate every input... Especially from Intents

Use exploit mitigations: ASLR, NX, PIE, SELinux

SECTION 5

PERFORMANCE & WHATS NEXT

WHATS NEXT??

- Keep mapping Android attack surfaces
- Build safer apps & know how they break
- Turn bugs → responsible CVEs → reputation
- Research new vectors!
 - Kernal space, JNI, Bluetooth, IoT
- Share findings, teach others, keep learning





“Stay strapped or get
clapped”

– *George Washingmachine*

THANK YOU



—
GET IN TOUCH ↘



+7 (499) 244-16-06



[WWW.LINKEDIN.COM/IN/DACKSESSIONS](https://www.linkedin.com/in/dacksessions)



DACKSESSIONS@PROTONMAIL.COM



_Jack.34

RAP INTERLUDE

